

Searching Techniques: Linear Search & Binary Search — Data Structures

Definition: Searching is the algorithmic process of locating the position of a target key element K within a collection of items (such as an array). The two primary searching techniques are **Linear Search** and **Binary Search**.

1. Detailed Technical Explanation

1. Linear Search (Sequential Search)

- **Concept:** Sequentially inspects every element in the array from index 0 to $N - 1$ until the target element is found or the end of array is reached.
- **Requirement:** Works on both **sorted and unsorted** arrays.
- **Time Complexity:** Best: $O(1)$, Average: $O(N)$, Worst: $O(N)$.

```
int linearSearch(int arr[], int n, int key) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == key)  
            return i; // Element found at index i  
    }  
    return -1; // Not found  
}
```

2. Binary Search (Divide and Conquer)

- **Concept:** Repeatedly divides the search interval in half by comparing the target key with the middle element $arr[mid]$:
- If $arr[mid] == key$: Target found!
- If $key < arr[mid]$: Search in left half ($high = mid - 1$).
- If $key > arr[mid]$: Search in right half ($low = mid + 1$).
- **Strict Prerequisite:** The array **MUST be sorted**.
- **Time Complexity:** Best: $O(1)$, Average: $O(\log N)$, Worst: $O(\log N)$.

```
// Iterative Binary Search Implementation
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2; // Prevents integer overflow
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1; // Not found
}
```

2. Comparison: Linear Search vs Binary Search

Feature	Linear Search	Binary Search
Array Prerequisite	Any array (Unsorted or Sorted).	Strictly Sorted Array Required.
Worst-Case Time	$O(N)$ (Linear)	$O(\log N)$ (Logarithmic)
Access Paradigm	Sequential access (Works on Linked Lists).	Random access (Requires direct indexing <code>arr[mid]</code>).
Comparisons for N=1000	Up to 1000 comparisons.	At most 10 comparisons ($2^{10} = 1024$).

3. Quick Recall Flow

Linear Search: Unsorted Arrays -> $O(N)$ | Binary Search: Sorted Array -> Halve Interval $mid = (low+high)/2$ -> $O(\log N)$